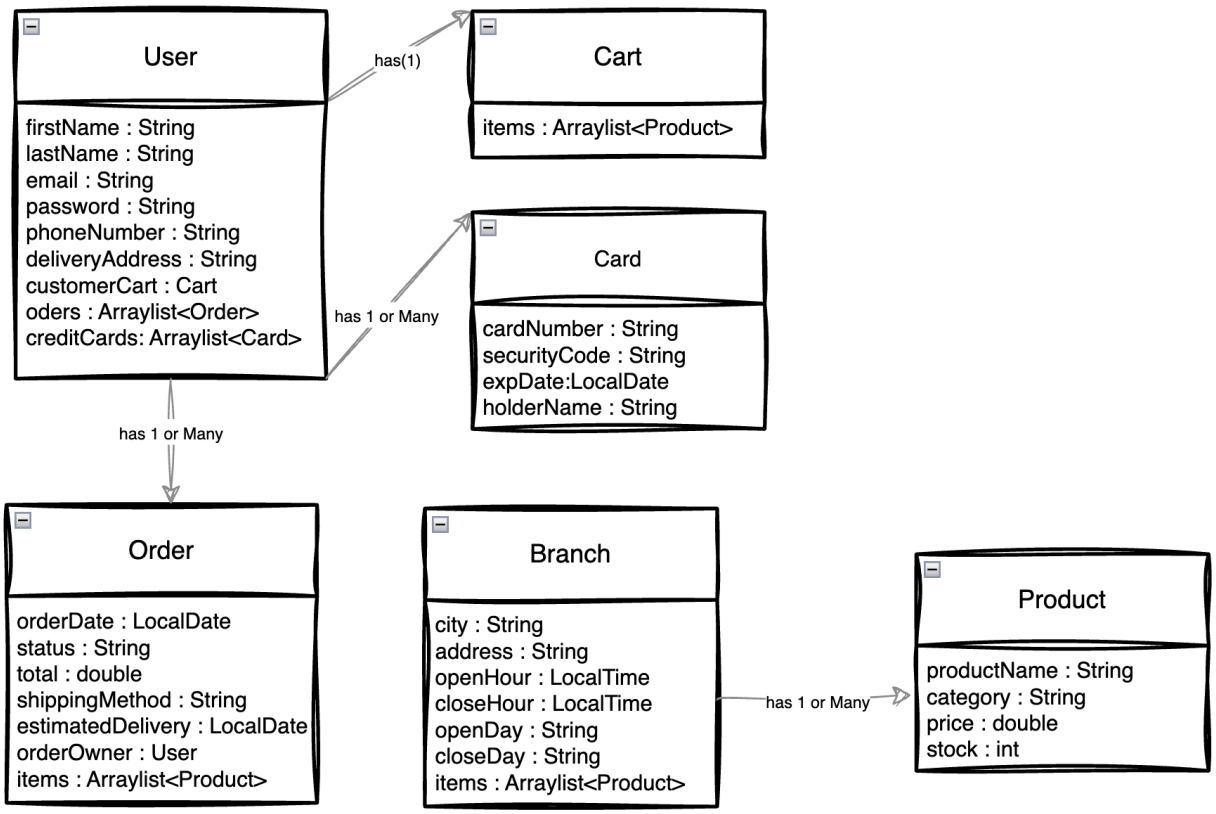
This system is an e-commerce platform inspired by Jarir Bookstore. Users can browse products from different store branches, add them to a cart, and complete a purchase by placing an order and paying online.

The system has six main classes. User stores the customer's personal and delivery info, and each user has one Cart, a list of past Orders, and saved payment Cards. Product holds each item's name, category, price, and stock, and is linked to Branch, which represents the store's different locations. Order is created after checkout and includes the ordered products, total cost, shipping details, and a status that also reflects the order status.

Models:


User
firstName : String
lastName : String
email : String
password : String
phoneNumber : String
deliveryAddress : String
customerCart : Cart
oders : Arraylist<Order>
creditCards: Arraylist<Card>
has(1)
Cart
items : Arraylist<Product>
has 1 or Many
Card
cardNumber : String
securityCode : String
expDate:LocalDate
holderName : String
has 1 or Many
Order
orderDate : LocalDate
status : String
total : double
shippingMethod : String
estimatedDelivery : LocalDate
orderOwner : User
items : Arraylist<Product>
Branch
city : String
address : String
openHour : LocalTime
closeHour : LocalTime
openDay : String
closeDay : String
items : Arraylist<Product>
has 1 or Many
Product
productName : String
category : String
price : double
stock : int

Validations for each Model:

User Model:

```
@NotEmpty(message = "First Name Cannot be EMPTY !")
@Size(min=2,max=15, message = "First Name should be between 2 to 15")
@Pattern(regexp = "^[a-zA-Z]+$", message = "Name must contain only letters
(no numbers or Special Chars)")
```

firstName

```
@NotEmpty(message = "Last Name Cannot be EMPTY !")
@Size(min=2,max=15, message = "Last Name should be between 2 to 15")
@Pattern(regexp = "^[a-zA-Z]+$", message = "last Name must contain only
letters (no numbers or Special Chars)")
```

lastName

```
@NotEmpty(message = "Email Cannot be EMPTY !")
@Email(message = "Your email is invalid")
```

email

```
@NotEmpty(message = "Phone number Cannot be EMPTY !")
@Pattern(regexp = "^\\d{10}$", message = "Phone number must be exactly 10
digits and contain only numbers")
```

phoneNumber

```
@NotEmpty(message = "Password cannot be empty!")
@Size(min = 8, message = "Password must be at least 8 characters long")
@Pattern(regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*[^a-zA-Z0-9]).+$", message =
"Password must contain at least one uppercase letter, one lowercase letter,
and one special character")
```

Password

```
@NotEmpty(message = "Address Cannot be EMPTY !")
```

deliveryAddress

Order Model:

```
@NotNull(message = "orderDate Cannot be EMPTY !")
@FutureOrPresent(message = "The order date must be today or in the future.")
```
orderDate

```
@NotEmpty(message = "status Cannot be EMPTY !")
@Pattern(regexp = "prepared|shipped|delivered", message = "Order status
should be prepared OR shipped OR delivered")
```
status

```
@NotNull(message = "total cannot be empty")
@Positive(message = "Total should be more than 0 ")
```
total

```
@NotEmpty(message = "shipping method Cannot be EMPTY !")
@Pattern(regexp = "pickUpFromBranch|delivery ", message = "choose
pickUpFromBranch OR delivery ")
```
shippingMethod

```
@NotNull(message = " estimated delivery Cannot be EMPTY !")
@FutureOrPresent(message = "estimated delivery date must be today or in the
future.")
```
estimated delivery

Branch Model:

```
@NotEmpty(message = "city Cannot be EMPTY !")
@Size(min=2 ,message = "city more than 2 letters")
@Pattern(regexp = "^[a-zA-Z]+$", message = "city must contain only letters
(no numbers or Special Chars)")
```

city

```
@NotEmpty(message = "Address Cannot be EMPTY !")
```

address

```
@NotNull(message = "Open Hour Cannot be EMPTY !")
@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "HH:mm:ss")
```

openHour

```
@NotNull(message = "Close Hour Cannot be EMPTY !")
@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "HH:mm:ss")
```

closeHour

```
@NotEmpty(message = "Open Day Cannot be EMPTY !")
@Pattern(regexp = "Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday",
message = " Open Day  should be from monday To sunday ")
```

openDay

```
@NotEmpty(message = " Open Day Cannot be EMPTY !")
@Pattern(regexp = "Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday",
message = " Open Day  should be from monday To sunday")
```

closeDay

Product Model:

```
@NotEmpty(message = "Product Name Cannot be EMPTY !")
@Size(min=2, message = " Product Name more than 2 letters")
```
productName

```
@NotEmpty(message = "you have to categorize the product")
@Pattern(regexp="Electronics|Computers|Books|VideoGames|OfficeSupplies|School
Supplies|Toys", message = " choose from existing cetegories")
```
category

```
@NotNull(message = "price cannot be empty")
@Positive(message = " price should be more than 0 ")
```
price

```
@NotNull(message = "stock cannot be empty")
@PositiveOrZero(message = " stock should be more than 0 ")
```
stock

Card Model:

```
@NotEmpty(message = "card number Cannot be EMPTY !")
@Size(min = 16, max = 19, message = "The number must be between 16 and 19
digits long.")
@Pattern(regexp = "^[0-9]*$", message = "card number must contain digits
only.")
```

cardNumber

```
@NotEmpty(message = "CVV Cannot be EMPTY !")
@Size(min = 3, max = 3, message = "Security Code must be 3 digits only.")
@Pattern(regexp = "^\\d+$", message = "card CVV must contain digits only.")
```

securityCode

```
@NotNull(message = "Expire Date Cannot be EMPTY !")
@FutureOrPresent(message = "Expire Date date must be today or in the
future.")
```

expDate

```
@NotEmpty(message = "Holder Name Cannot be EMPTY !")
@Size(min=2, message = "Holder Name should be more than 2 letters")
@Pattern(regexp = "^[a-zA-Z ]+$" ,message = "holder Name must contain only
letters (no numbers or Special Chars)")
```

holderName